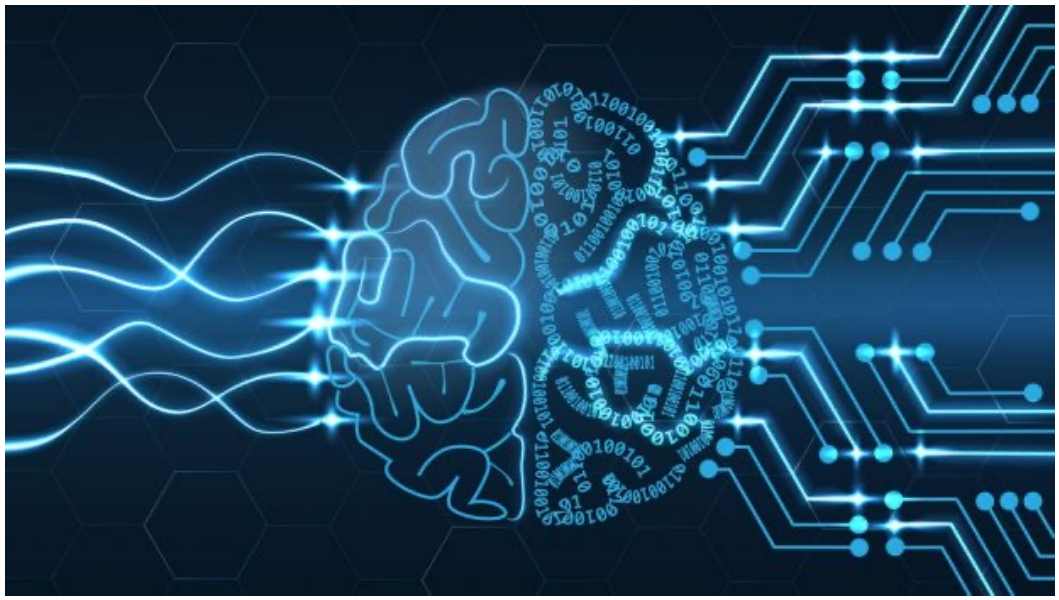


Chapter 4: Deep Learning

Changcheng Huang
Department of Systems and Computer Engineering

Outline

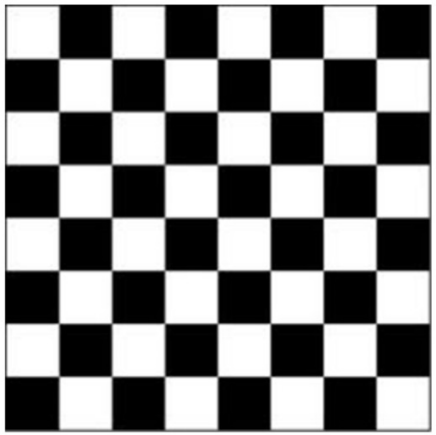
- Motivation
- Deep Feedforward Networks
- Training and Regularization
- Optimization for Training

Outline

- Motivation
- Deep Feedforward Networks
- Training and Regularization
- Optimization for Training

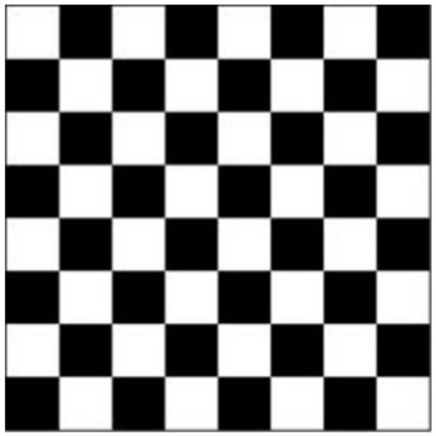
Challenge

- A region is a neighborhood that can be predicted accurately using simple mechanisms such as logistic regression or linear regression
- Challenge
  - Use $O(k)$ examples to define $O(2^k)$ regions



Solution

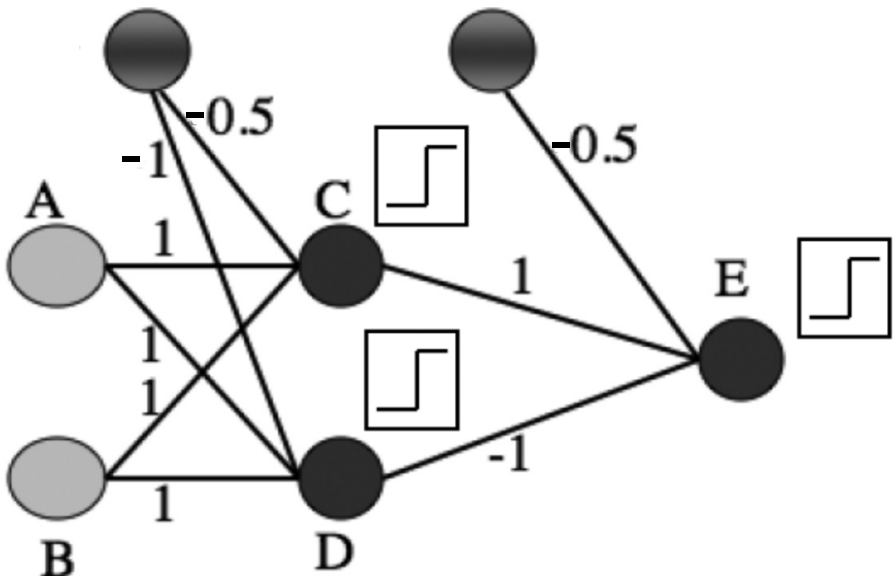
- Solution
  - Introduce dependencies between regions
  - Make assumptions reasonable for broad applications
- Deep learning
  - assume data is generated by composition of features at multiple levels in a hierarchy

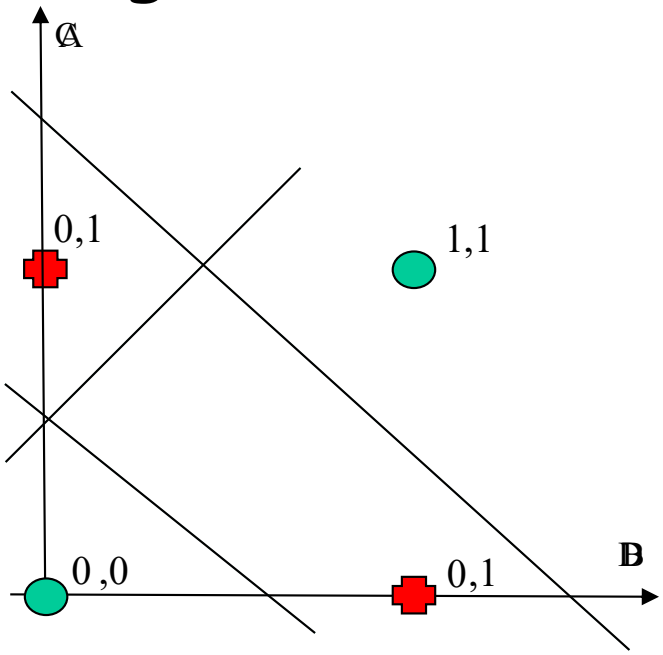


Example

- Cannot be separated by logistic regression

A	B	E
0	0	0
0	1	1
1	0	1
1	1	0





A layered decision process

Outline

- Motivation
- Deep Feedforward Networks
- Training and Regularization
- Optimization for Training

Definitions

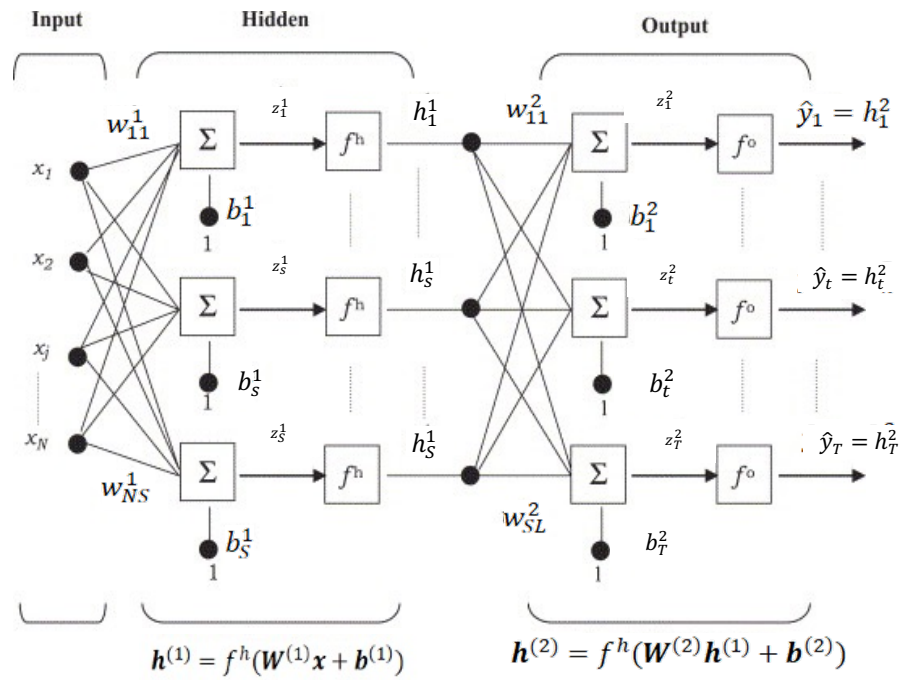
- Formulation

$$f(X,\theta) = f^{(3)}(f^{(2)}\left(f^{(1)}(X,\theta_1),\theta_2\right),\theta_3)$$

- Each function is a layer
- Hidden layers: no training data for output
- Width: dimensionality of hidden layers
- Depth: the number of layers
- Choosing function: linear + nonlinear

General Neural Network (GNN)

- Use generic function
- A linear function
  - Dimensionality
  - Also called affine transform
- A nonlinear activation function
- Output layer
  - Probability model preferred



Output Units for Regression

- Need a probability model for MLE
  - Ideally contains exponential function
  - Good derivative property
- Gaussian distributions

$$p(Y|H) = \mathcal{N}(Y; z, I)$$
$$z = w^T \boldsymbol{h} + b$$

  - Learning variance is expensive and not useful
  - Linear units do not saturate
  - MLE Cost function

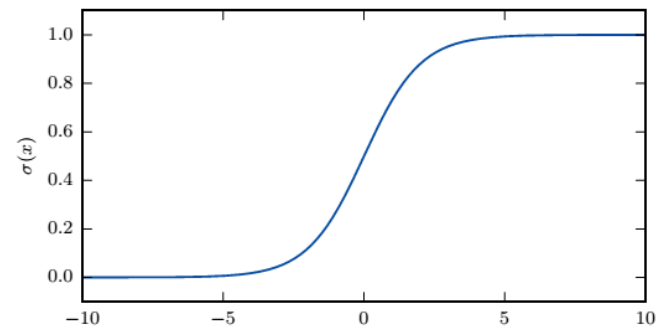
$$J = (y - z)^2$$
$$z = w^T \boldsymbol{h} + b$$

Binary Output Units

- Use sigmoid units for Bernoulli

$$p(Y|H) = \frac{\exp(YZ)}{1+\exp(Z)} = \sigma((2Y-1)Z)$$

$$Z = w^T H + \mathrm{b}$$



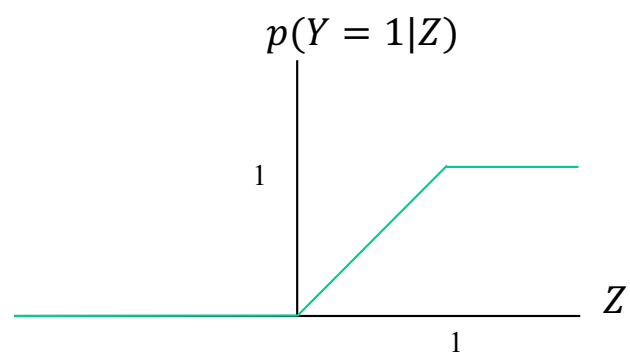
- Use linear for Bernoulli

$$p(Y=1|Z) = \max\{0, \min\{1, Z\}\}$$

  - Gradient becomes 0 outside range

$$Z = w^T H + \mathrm{b}$$

$$p(Y=1|H) = \max\{0, \min\{1, w^T H + b\}\}$$



Multinoulli Output Units

- Assume $z_k = w_k^T \boldsymbol{h} + b_k,\ k = 1, \cdots, K$

  then $p(Y = k|\boldsymbol{h}) = \frac{\exp(z_k)}{\sum_{k'} \exp(z_{k'})} = \mathrm{softmax}(z_k)$

- Softmax function saturates when the differences between input values are very large

Hidden Units

- Rectified Linear Units (ReLU)

$$H_i = g(Z_i) = \max\{0, Z_i\}, Z_i = w_i^T X + b_i$$

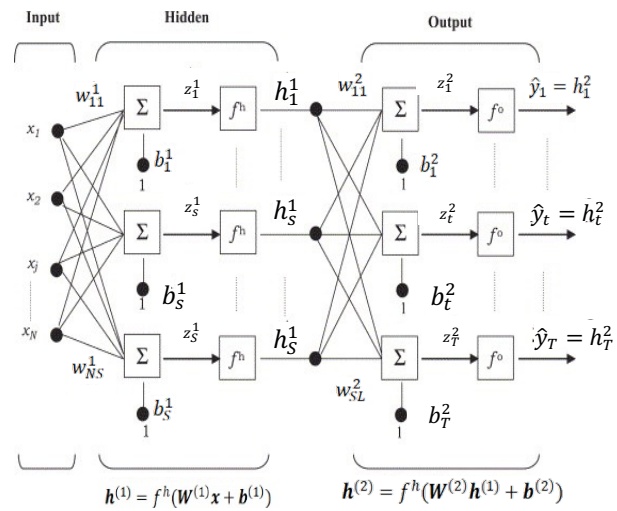
- Generalization

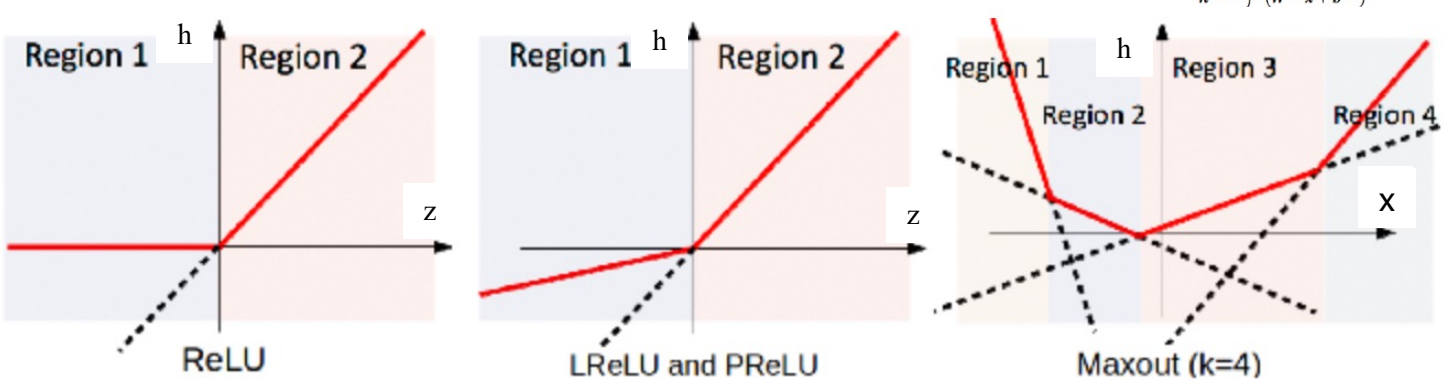
$$H_i = g(Z, \alpha)_i = \max(0, Z_i) + \alpha_i \min(0, Z_i)$$

- Maxout Units
  - Divide components of Z into K groups

$$H_i = g(Z)_i = \max_{j \in \mathbb{G}_i} Z_j$$

- Logistic sigmoid function
  - Less used due to saturation





Other Activation Functions

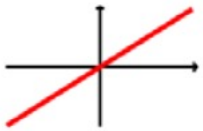
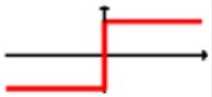
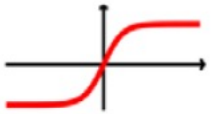
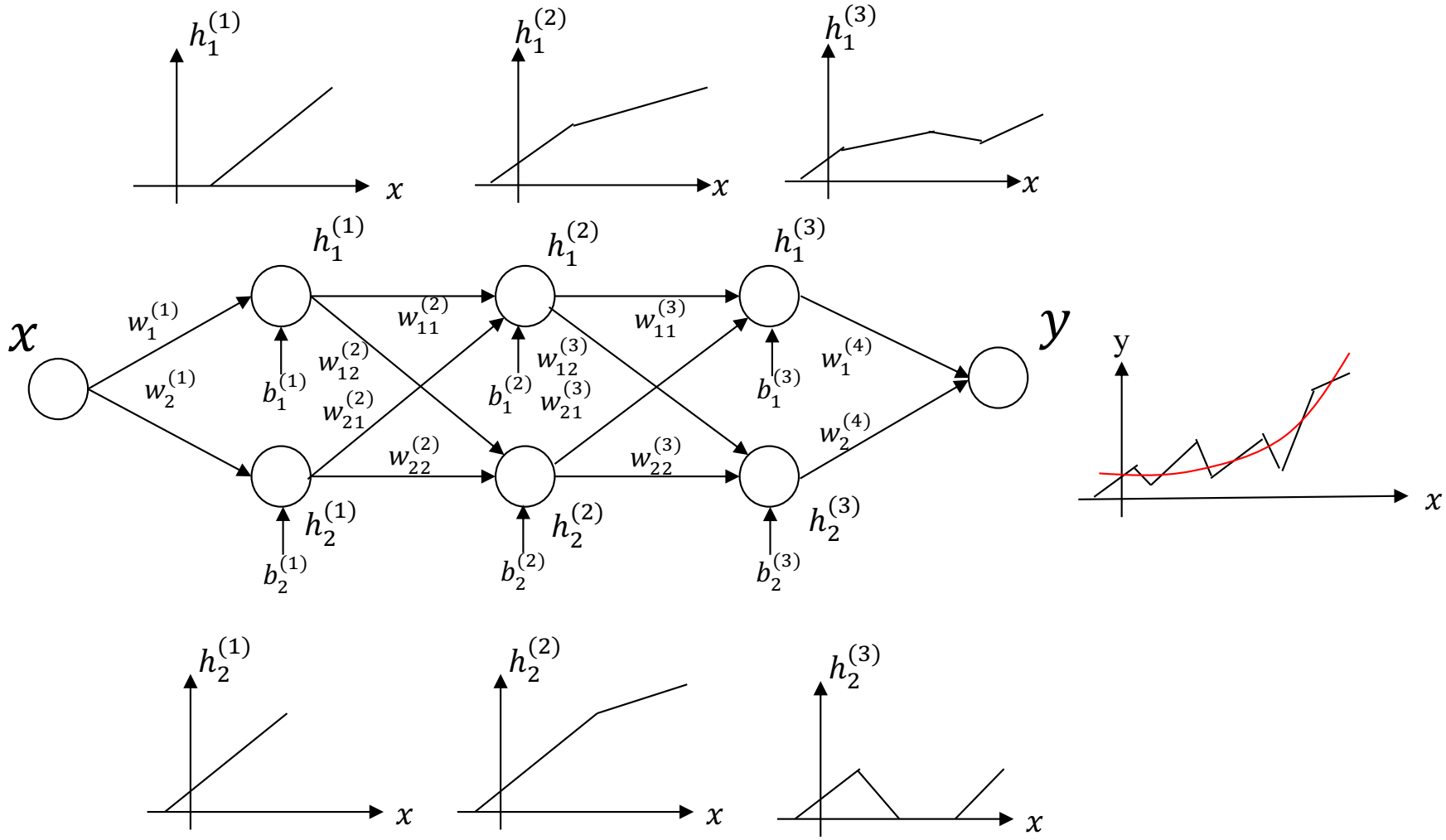
Activation function	Equation	Example	1D graph
Linear	$\phi(z) = z$	Adaline, linear regression	
Unit step (Heaviside function)	$\phi(z) = \begin{cases} 0 & z < 0 \\ 0.5 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Sign (signum)	$\phi(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Piece-wise linear	$\phi(z) = \begin{cases} 0 & z \le -\frac{1}{2} \\ z + \frac{1}{2} & -\frac{1}{2} \le z \le \frac{1}{2} \\ 1 & z \ge \frac{1}{2} \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, multilayer NN	
Hyperbolic tangent (tanh)	$\phi(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Multilayer NN, RNNs	
ReLU	$\phi(z) = \begin{cases} 0 & z < 0 \\ z & z > 0 \end{cases}$	Multilayer NN, CNNs	

Illustration for Impacts of Multiple Layers



Universal Approximation Theorem

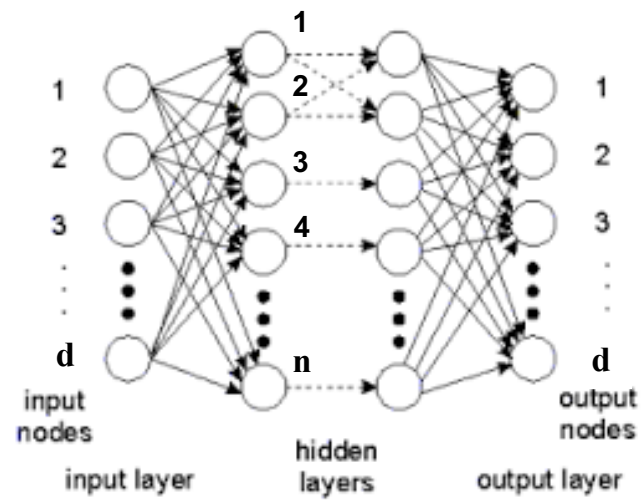
- Linear output
- At least one hidden layer with nonlinear activation
- Enough number of hidden units
- Can approximate any Borel measureable functions and its derivatives

Architecture Design

- Layers in a chain

$$\boldsymbol{h}_1 = g(w_1^T \boldsymbol{x} + b_1)$$

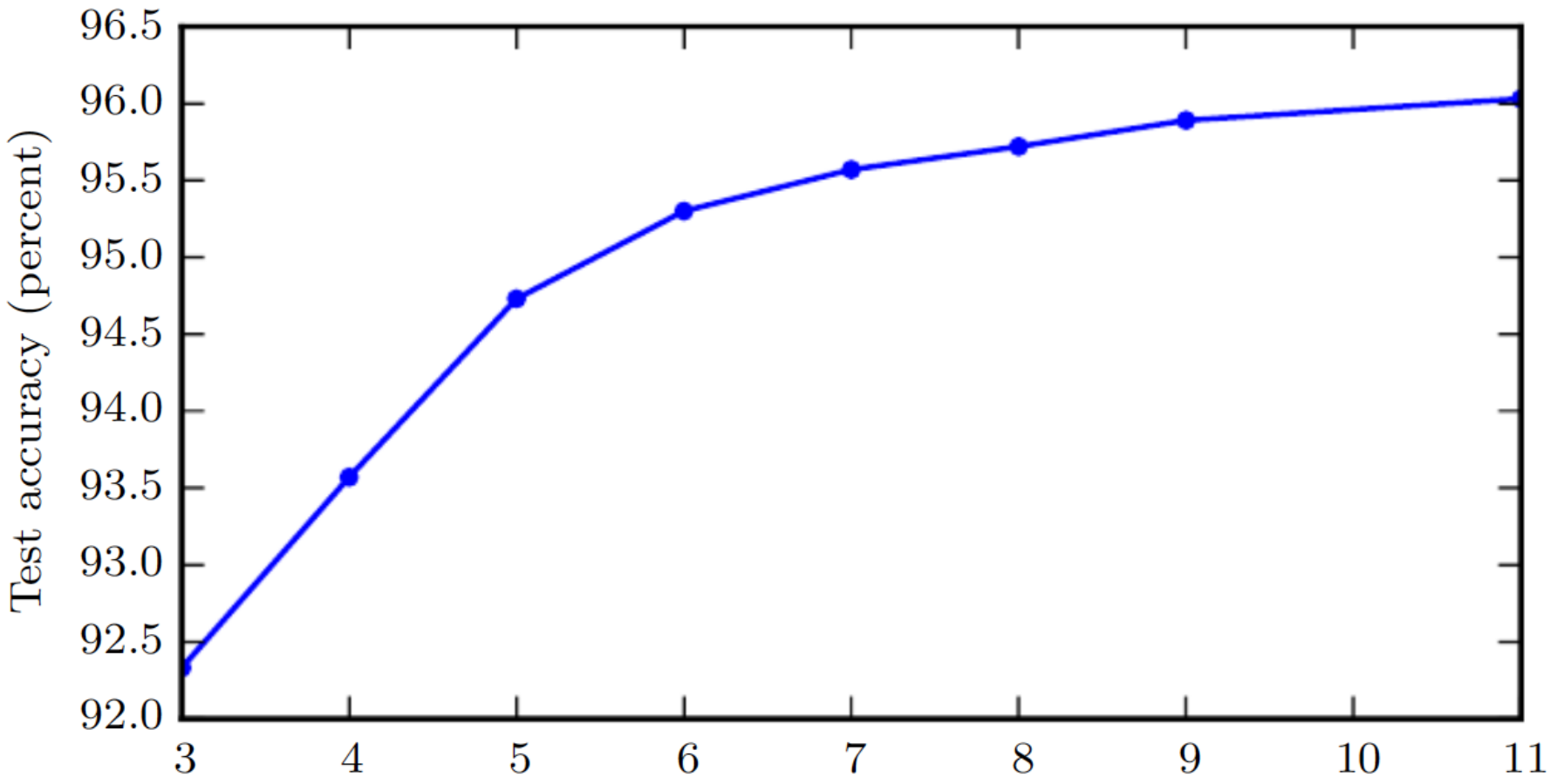
$$\boldsymbol{h}_{k+1} = g(w_2^T \boldsymbol{h}_k + b_2)$$



- Depth and width
  - Shallow networks need exponential number of units
  - Number of regions increases exponential with depth
  - d inputs, depth l, n units per hidden layer
  - Number of linear regions represented

$$O\left(\binom{n}{d}^{d(l-1)} n^d\right)$$

Impacts of Depth



Outline

- Motivation
- Deep Feedforward Networks
- Training and Regularization
- Optimization for Training

Cost Function

- Parameter Norm Penalties

$$J(\theta; \boldsymbol{x}, y) = L(\theta; \boldsymbol{x}, y) + \lambda\Omega(\theta)$$

- L^2 Parameter Regularization

$$J(w, b; \boldsymbol{x}, y) = \frac{\lambda}{2} w^T w + \frac{\lambda}{2} b^T b + L(w, b; \boldsymbol{x}, y)$$

$$\nabla_w J(w; \boldsymbol{x}, y) = \lambda w + \nabla_w L(w, b; \boldsymbol{x}, y)$$

$$w \leftarrow w - \epsilon(\lambda w + \nabla_w L(w, b; \boldsymbol{x}, y))$$

$$w \leftarrow (1 - \epsilon\lambda) w - \epsilon \nabla_w L(w, b; \boldsymbol{x}, y)$$

  - Multiplicatively shrink the weight vector

Forward Propagation Generalization

$$\boldsymbol{z}^{(1)} = W^{(1)}\boldsymbol{x}+\boldsymbol{b}^{(1)}$$

$$h_s^{(1)} = f^h\left(z_s^{(1)}\right) = f^h(\boldsymbol{z}^{(1)})_s$$

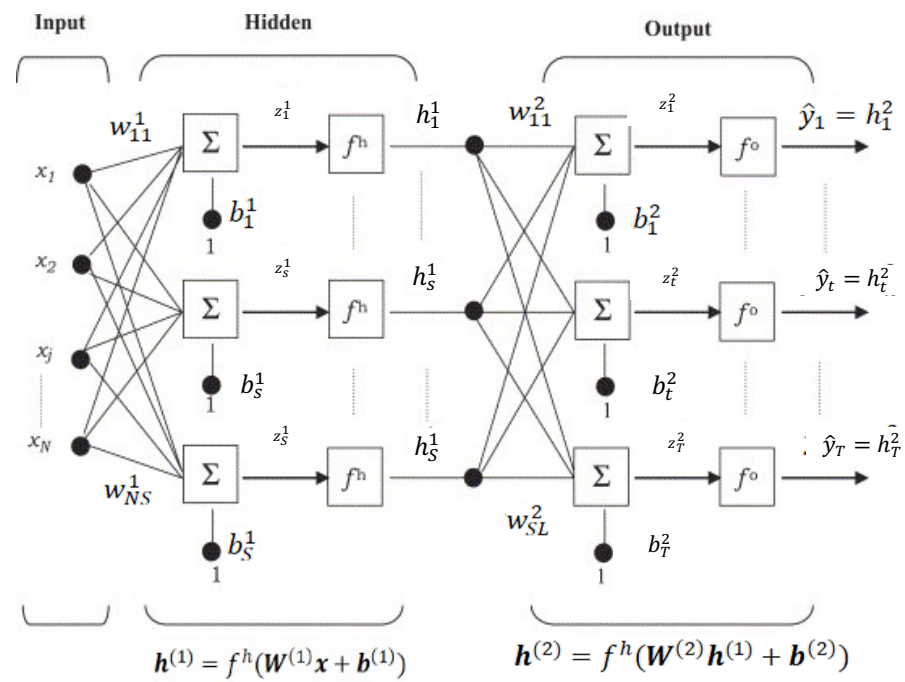
$$\boldsymbol{h}^{(1)} = f^h(\boldsymbol{z}^{(1)})$$

$$\boldsymbol{z}^{(k)} = W^{(k)}\,\boldsymbol{h}^{(k-1)} + \boldsymbol{b}^{(k)}$$

$$\boldsymbol{h}^{(k)} = f^h(\boldsymbol{z}^{(k)})$$

$$\hat{\boldsymbol{y}} = f^o(\boldsymbol{z}^{(l)})$$

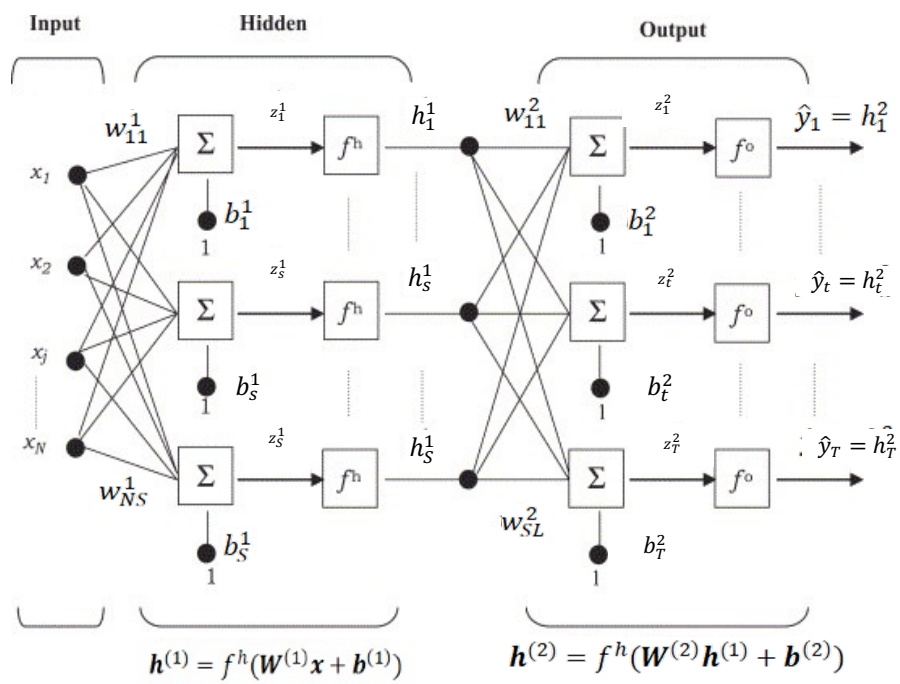
$$\hat{\boldsymbol{y}} = f^o(W^{(l)}f^h(W^{(l-1)}f^h(\cdots)+\boldsymbol{b}^{(l-1)})+\boldsymbol{b}^{(l)})$$



Training: Forward Propagation

Require: Network depth, l
Require: $\boldsymbol{W}^{(i)}, i \in \{1, \ldots, l\}$, the weight matrices of the model
Require: $\boldsymbol{b}^{(i)}, i \in \{1, \ldots, l\}$, the bias parameters of the model
Require: $\boldsymbol{x}$, the input to process
Require: $\boldsymbol{y}$, the target output

$\boldsymbol{h}^{(0)} = \boldsymbol{x}$
for $k = 1, \ldots, l$ **do**
  $z^{(k)} = \boldsymbol{b}^{(k)} + \boldsymbol{W}^{(k)} \boldsymbol{h}^{(k-1)}$
  $\boldsymbol{h}^{(k)} = f(z^{(k)})$
end for
$\hat{\boldsymbol{y}} = \boldsymbol{h}^{(l)}$
$J = L(\hat{\boldsymbol{y}}, \boldsymbol{y}) + \lambda \Omega(\theta)$

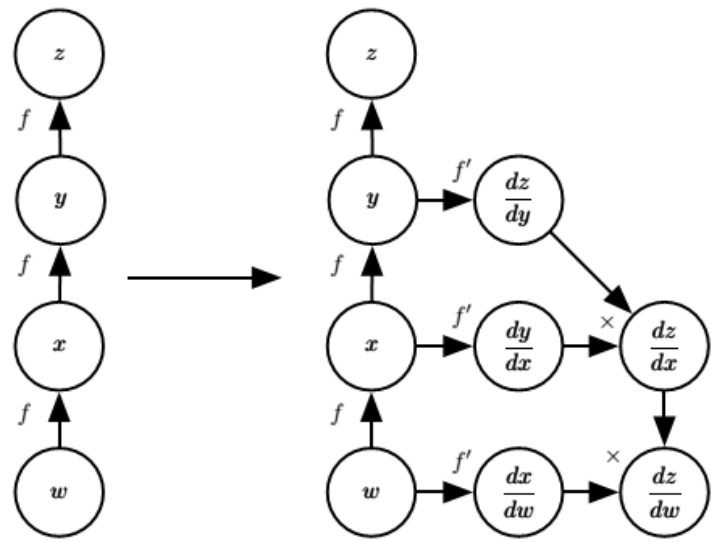


Chain Rule of Calculus

- Chain rule of calculus

$$z = f(y) = f\big(f(x)\big) = f(f(f(w)))$$

$$\frac{dz}{dw} = \frac{dz}{dy}\frac{dy}{dx}\frac{dx}{dw}$$

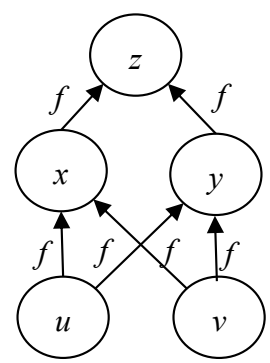


- Multivariable chain rule

$$z = f(x, y) = f(f(u, v), f(u, v))$$

$$\frac{\partial z}{\partial u} = \frac{\partial z}{\partial x}\frac{\partial x}{\partial u} + \frac{\partial z}{\partial y}\frac{\partial y}{\partial u}$$

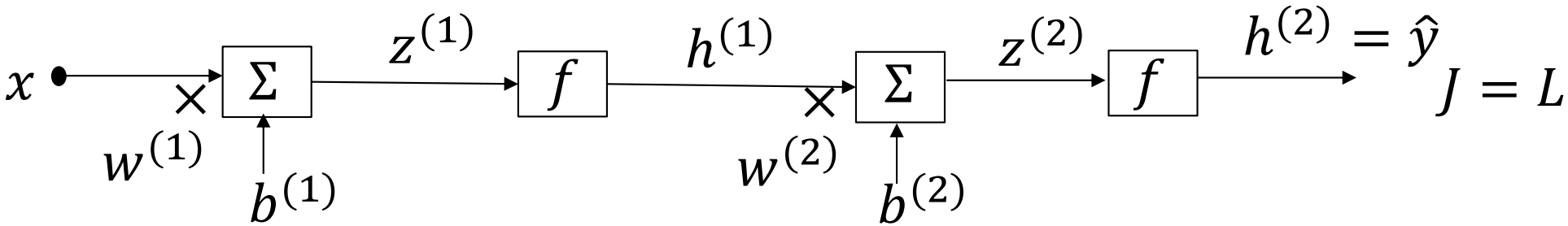
$$\frac{\partial z}{\partial v} = \frac{\partial z}{\partial x}\frac{\partial x}{\partial v} + \frac{\partial z}{\partial y}\frac{\partial y}{\partial v}$$



$$\nabla_{u,v} z = J_{x,y}(u, v)^{\intercal} \nabla_{x,y} z$$

$$J_{f \circ g}(\boldsymbol{x}) = J_f(g(\boldsymbol{x}))J_g(\boldsymbol{x})$$

Training for One Dimension



Goal: $\nabla_{\boldsymbol{b}^{(k)}} J$, $\nabla_{\boldsymbol{w}^{(k)}} J$

$$g = \nabla_{\boldsymbol{h}^{(2)}} J = \frac{dJ}{dh^{(2)}} = \frac{d(y-\hat{y})^2}{d\hat{y}} = -2(\mathrm{y}-\hat{y})$$

$$g \leftarrow \nabla_{\boldsymbol{z}^{(2)}} J = g f'\left(z^{(2)}\right) = g = -2(\mathrm{y}-\hat{y})$$

$$\nabla_{\boldsymbol{b}^{(2)}} J = g \nabla_{\boldsymbol{b}^{(2)}} z^{(2)} = -2(\mathrm{y}-\hat{y})$$

$$\nabla_{W^{(2)}} J = g \nabla_{W^{(2)}} z^{(2)} = g h^{(1)} = -2(\mathrm{y}-\hat{y}) h^{(1)}$$

$$g \leftarrow \nabla_{\boldsymbol{h}^{(1)}} J = g \nabla_{\boldsymbol{h}^{(1)}} z^{(2)} = g w^{(2)} = -2(\mathrm{y}-\hat{y}) w^{(2)}$$

$$g \leftarrow \nabla_{\boldsymbol{z}^{(1)}} J = g f'\left(z^{(1)}\right) = -2(\mathrm{y}-\hat{y}) w^{(2)}$$

$$\nabla_{\boldsymbol{b}^{(1)}} J = g \nabla_{\boldsymbol{b}^{(1)}} z^{(1)} = g = -2(\mathrm{y}-\hat{y}) w^{(2)}$$

$$\nabla_{W^{(1)}} J = g \nabla_{W^{(1)}} z^{(1)} = g x = -2(\mathrm{y}-\hat{y}) w^{(2)} x$$

Special Vector Operations

- Consider two vectors

$$\boldsymbol{a} = \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix}, \boldsymbol{b} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix}$$

- Dyadic Product

$$\boldsymbol{a} \otimes \boldsymbol{b} = \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} \begin{pmatrix} b_1 & b_2 & b_3 \end{pmatrix} = \begin{pmatrix} a_1 b_1 & a_1 b_2 & a_1 b_3 \\ a_2 b_1 & a_2 b_2 & a_2 b_3 \\ a_3 b_1 & a_3 b_2 & a_1 b_3 \end{pmatrix}$$

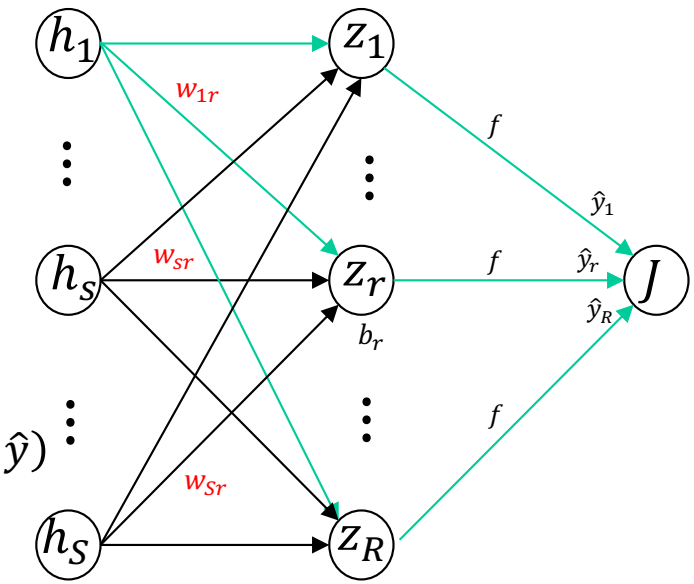
- Hadamard Product

$$\boldsymbol{a} \odot \boldsymbol{b} = \begin{pmatrix} a_1 b_1 \\ a_2 b_2 \\ a_3 b_3 \end{pmatrix}$$

Training for Multiple Dimensions

- Multiple variable chain rule

$$\frac{\partial J}{\partial h_1} = \sum_r \frac{\partial z_r}{\partial h_1}\frac{d\hat{y}_r}{dz_r}\frac{\partial J}{\partial \hat{y}_r} \qquad J_{\boldsymbol{h}}\boldsymbol{z} = \begin{pmatrix} \frac{\partial z_1}{\partial h_1} & \cdots & \frac{\partial z_R}{\partial h_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial z_1}{\partial h_S} & \cdots & \frac{\partial z_R}{\partial h_S} \end{pmatrix}$$



$$\nabla_{\boldsymbol{h}} J = J_{\boldsymbol{h}}(\boldsymbol{z})\nabla_{\boldsymbol{z}} J = J_{\boldsymbol{h}}(\boldsymbol{z}) f'(\boldsymbol{z}) \odot \nabla_{\hat{\boldsymbol{y}}} J = J_{\boldsymbol{h}}(\boldsymbol{z}) f'(\boldsymbol{z})(-2)(y - \hat{y})$$

- Given

$$z_r = \boldsymbol{w}_r^{\mathsf{T}}\boldsymbol{h} + b_r = w_{1r}h_1 + \cdots + w_{sr}h_s + \cdots + w_{Sr}h_S + b_r$$

Then

$$\frac{\partial z_r}{\partial h_s} = w_{sr}, \qquad \nabla_{\boldsymbol{h}} z_r = \boldsymbol{w}_r, \qquad \nabla_{\boldsymbol{h}}\boldsymbol{z} = W, \qquad \nabla_{\boldsymbol{h}} J = W\nabla_{\boldsymbol{z}} J$$

$$\frac{\partial z_r}{\partial w_{sr}} = h_s, \nabla_{\boldsymbol{w}_r} z_r = \boldsymbol{h}, \quad \frac{\partial J}{\partial w_{sr}} = h_s\frac{\partial J}{\partial z_r}, \ \nabla_{\boldsymbol{w}_r} J = \boldsymbol{h}\frac{\partial J}{\partial z_r}, \ \nabla_{\boldsymbol{W}} J = \boldsymbol{h} \otimes \nabla_{\boldsymbol{z}} J$$

$$\frac{\partial z_r}{\partial b_r} = 1, \qquad \nabla_{\boldsymbol{b}} z_r = \boldsymbol{I}, \qquad \nabla_{\boldsymbol{b}} J = \nabla_{\boldsymbol{z}} J$$

Training: Backward Propagation

- Given: $\boldsymbol{h}^{(k)} = f\left(\boldsymbol{z}^{(k)}\right), \boldsymbol{z}^{(k)} = W^{(k)}\boldsymbol{h}^{(k-1)} + \boldsymbol{b}^{(k)}, k = 1, \cdots, l, \boldsymbol{h}^{(l)} = \hat{\boldsymbol{y}}$
- Goal: $\nabla_{\boldsymbol{b}^{(k)}} J, \nabla_{\boldsymbol{w}^{(k)}} J$, where $J = L(\hat{\boldsymbol{y}}, y) + \lambda\Omega(W, B)$
- Need to find $\nabla_{\boldsymbol{z}^{(k)}} J, \nabla_{\boldsymbol{h}^{(k)}} J$ recursively
  - Start by $k = l$

    $\boldsymbol{g} = \nabla_{\boldsymbol{h}^{(l)}} J = \nabla_{\hat{\boldsymbol{y}}} J$
  - Add nonlinear function gradient

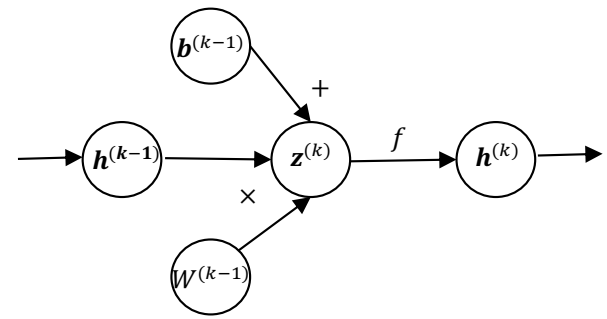
    $\boldsymbol{g} \leftarrow \nabla_{\boldsymbol{z}^{(k)}} J = \boldsymbol{g} \odot f'(\boldsymbol{z}^{(k)})$
  - Get gradients for layer k parameters

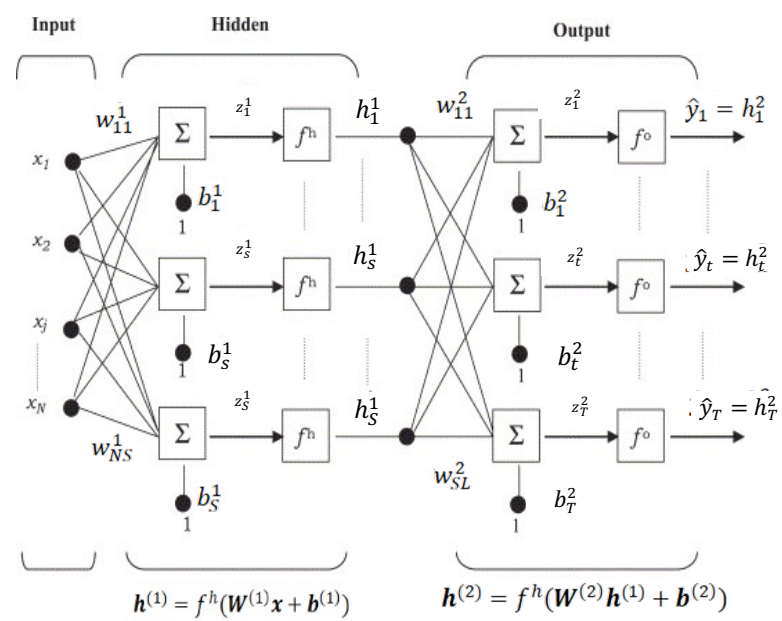
    $\nabla_{\boldsymbol{b}^{(k)}} J = \boldsymbol{g} + \lambda \nabla_{b^{(k)}} \Omega(W, B)$

    $\nabla_{W^{(k)}} J = \boldsymbol{h}^{(k-1)} \otimes \boldsymbol{g}^{\mathsf{T}} + \lambda \nabla_{W^{(k)}} \Omega(W, B)$
  - Propagate to the next layer

    $\boldsymbol{g} \leftarrow \nabla_{\boldsymbol{h}^{(k-1)}} J = W^{(k)}\boldsymbol{g}$

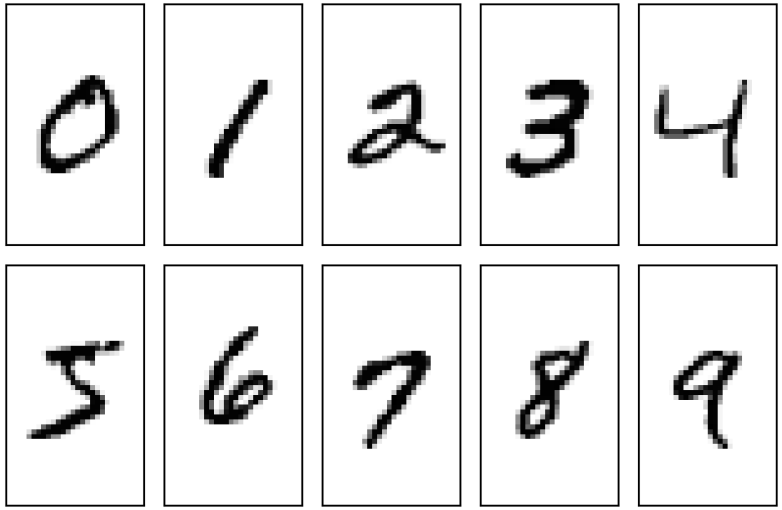
From the result of last slide

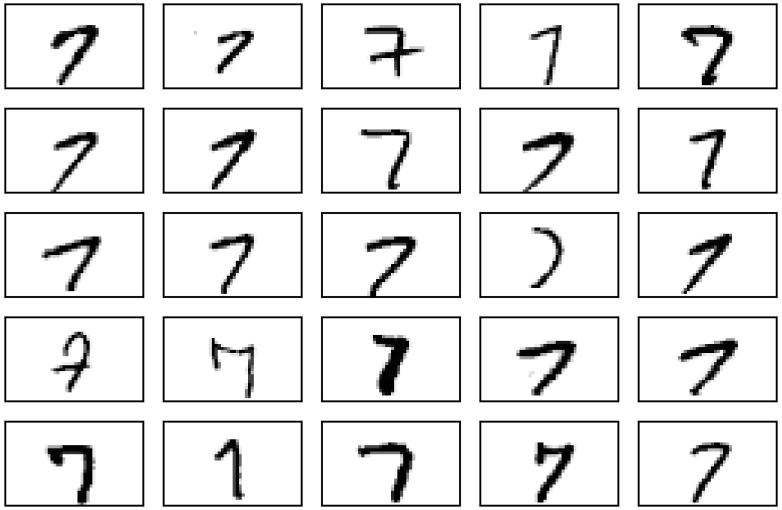




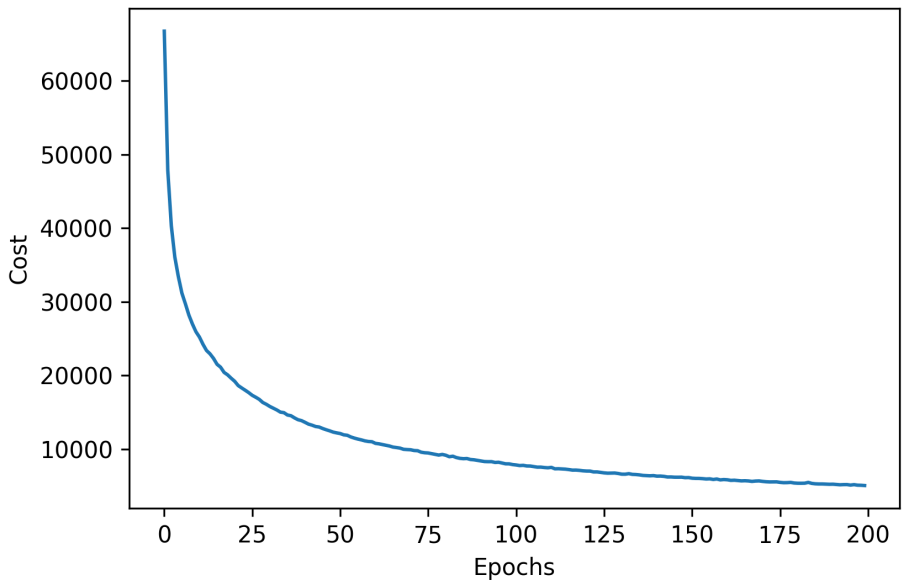
Example

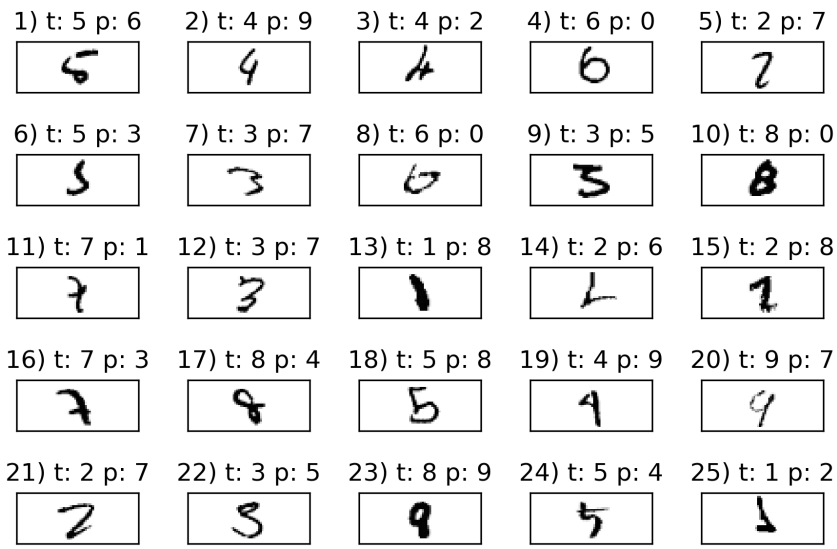
- MNIST dataset
  - Handwritten digits from 250 people
  - Each image consists of 28x28 pixels





Results


Cost
60000
50000
40000
30000
20000
10000
0
25
50
75
100
125
150
175
200
Epochs

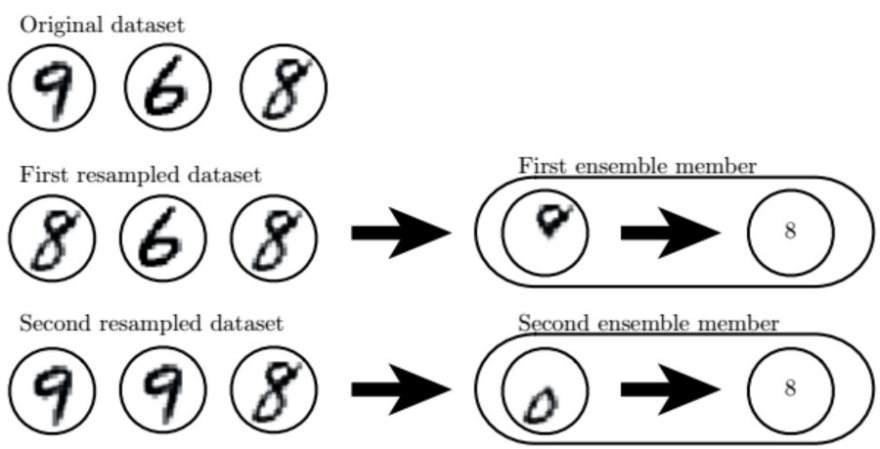

1) t: 5 p: 6
2) t: 4 p: 9
3) t: 4 p: 2
4) t: 6 p: 0
5) t: 2 p: 7
6) t: 5 p: 3
7) t: 3 p: 7
8) t: 6 p: 0
9) t: 3 p: 5
10) t: 8 p: 0
11) t: 7 p: 1
12) t: 3 p: 7
13) t: 1 p: 8
14) t: 2 p: 6
15) t: 2 p: 8
16) t: 7 p: 3
17) t: 8 p: 4
18) t: 5 p: 8
19) t: 4 p: 9
20) t: 9 p: 7
21) t: 2 p: 7
22) t: 3 p: 5
23) t: 8 p: 9
24) t: 5 p: 4
25) t: 1 p: 2

Regularization

- Dataset Augmentation
  - More data means better generalization
    - Solution: Fake data
      - Image recognition: rotating an image
      - Speech recognition: inject noise
- Parameter Tying
  - Explore prior knowledge on related parameters
  - e.g., convolution network
- Sparse Representation
  - Limit or reduce nonzero features
- Use unsupervised learning help group data

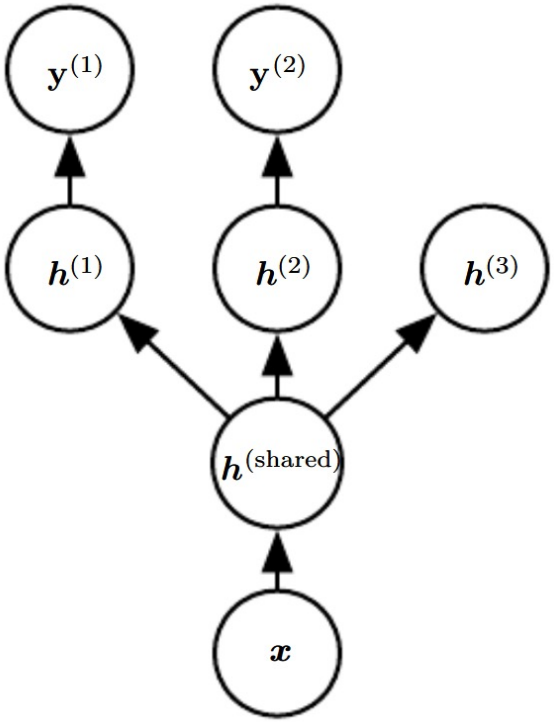
Regularization

- Bagging and Ensemble Methods
  - Also called bootstrap aggregation
  - Sample original dataset into multiple sets with replacements
  - Try a model on each set
  - Vote on testing data



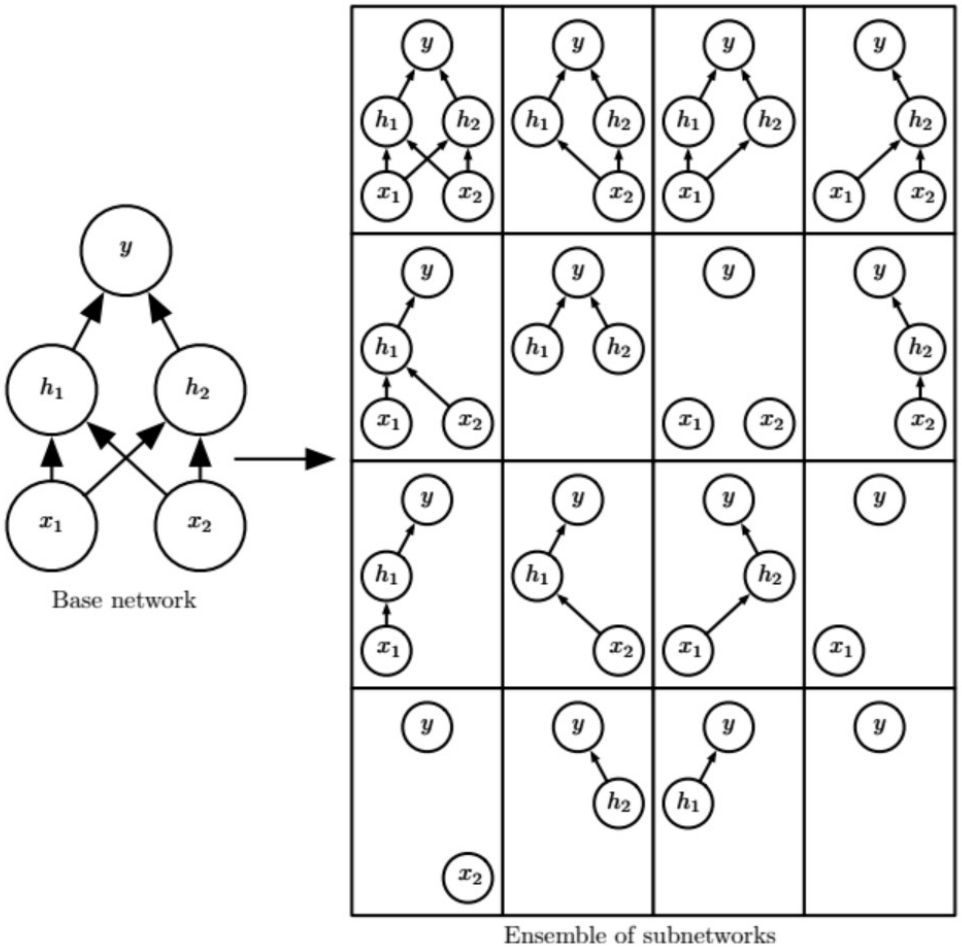
Regularization

- Noise Robustness
  - Add noise to hidden units
  - Add noise to weights
  - Add noise to outputs
- Multitask Learning
  - Pooling sample points out of several tasks



Regularization

- Dropout
  - Similar to bagging in using multiple models
  - Different from bagging by sharing parameters
  - For each minibatch, select a mask μ for model structure
  - Minimizing $\mathbb{E}_{\mu}(\theta, \mu)$



Regularization

- Adversarial training
  - Neural networks may fail due to technical problem in calculating gradient
  - Find cases that a model is likely to fail
  - Train the model with those cases

$+ .007 \times$ $=$

$\boldsymbol{x}$	$\text{sign}(\nabla_{\boldsymbol{x}} J(\boldsymbol{\theta}, \boldsymbol{x}, y))$	$\boldsymbol{x} + \epsilon\,\text{sign}(\nabla_{\boldsymbol{x}} J(\boldsymbol{\theta}, \boldsymbol{x}, y))$
$y =$"panda" w/ 57.7% confidence	"nematode" w/ 8.2% confidence	"gibbon" w/ 99.3% confidence

Bayesian Estimator

- If we are not sure θ is fixed
  - MLE not work
  - Assume prior probability distribution: $\theta \sim p(\theta)$
  - Still assume model $p(Y|X,\theta)$

$$p(Y|X) = \int p(Y|X,\theta)\ p(\theta)\mathrm{d}\theta$$

- Taking experience into account, use Bayes rule

$$p(\theta|y,\boldsymbol{x}) = \frac{p(y|\boldsymbol{x},\theta)p(\theta)}{\int p(y|\boldsymbol{x},\theta')p(\theta')d\theta'}$$

- Prediction

$$p(Y|X) = \int p(Y|X,\theta)\ p(\theta|y,\boldsymbol{x})\mathrm{d}\theta$$

- Often used as bound or benchmark

An example

- Consider a communication system

$$Y = X + N$$

$$X: \{0,1\}, \qquad N \sim \mathcal{N}(\theta, 1), \qquad \theta \in [0,1]$$

$$p(Y|X=0,\theta) \sim \mathcal{N}(\theta, 1), \qquad p(Y|X=1,\theta) \sim \mathcal{N}(1+\theta, 1)$$

- Assume prior distribution $\theta \sim \mathcal{U}(0,1)$
- Given an example $(x, y) = (1, 1.1)$
- Posterior distribution

$$p(\theta|y,\boldsymbol{x}) = \frac{p(y|\boldsymbol{x},\theta)p(\theta)}{\int p(y|\boldsymbol{x},\theta')p(\theta')d\theta'} = \frac{\exp(-\frac{(1.1-1-\theta)^2}{2})}{\int_0^1 \exp\left(-\frac{(1.1-1-\theta')^2}{2}\right)d\theta'}$$

- Prediction

$$p(Y|X=0) = \int p(Y|X=0,\theta)\, p(\theta|y,\boldsymbol{x})\mathrm{d}\theta$$

$$= \int_0^1 \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(Y-\theta)^2}{2}\right) p(\theta|y,\boldsymbol{x})d\theta$$

Outline

- Motivation
- Deep Feedforward Networks
- Training and Regularization
- Optimization for Training

Optimization for Training

- Training is an optimization process

$$\theta_{ML} = \underset{\theta}{\operatorname{argmax}} \sum_{i=1}^{M} \log p_{\text{model}}(\boldsymbol{x}, y; \theta)$$

  - Standard error goes down by $\sigma/\sqrt{M}$
  - Inefficient to use entire training set
- Stochastic Gradient Descent
  - Also called online method
    - use a single sample point or a minibatch
  - Use the entire set
    - Inefficient
    - First epoch is unbiased estimator
    - Multi epochs: overfitting

Minibatch

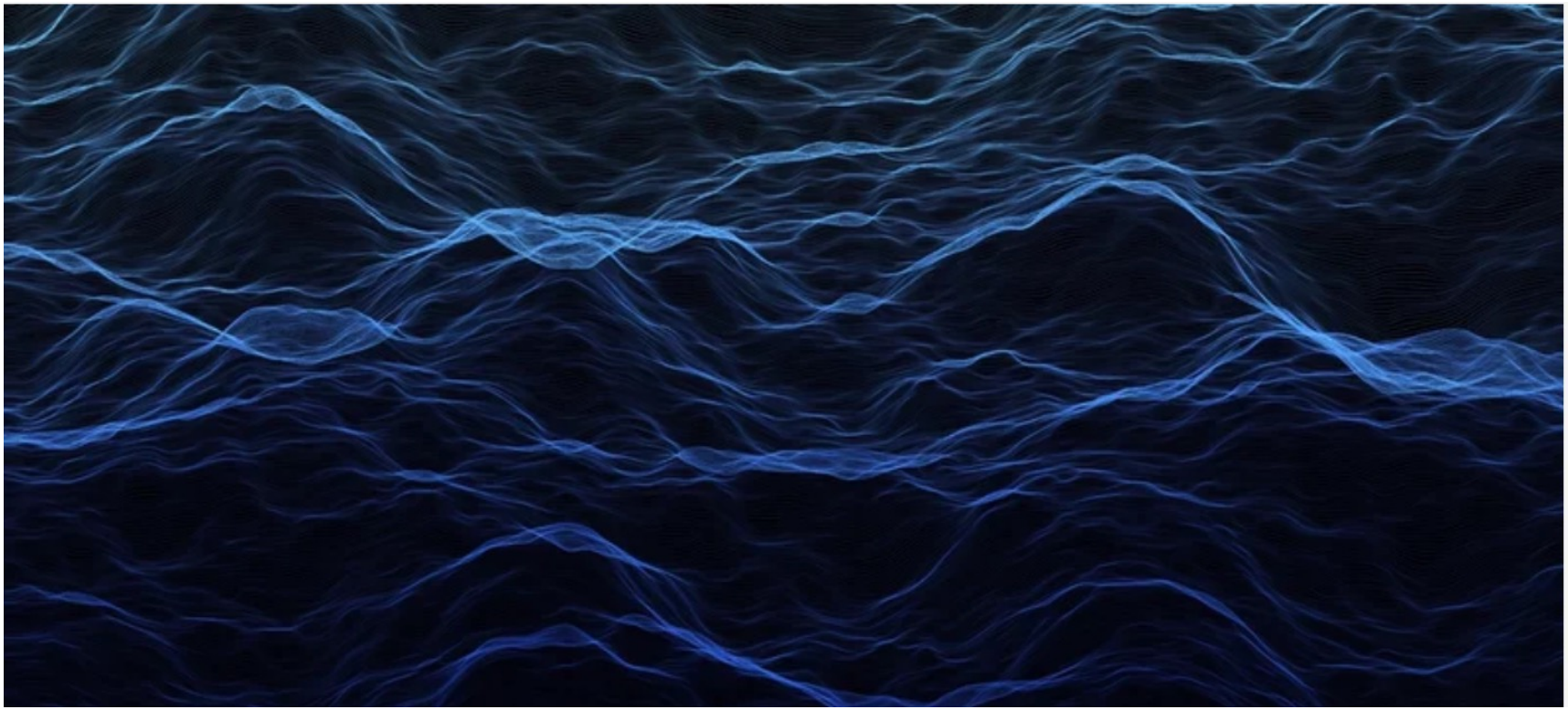
- Minibatch
  - Sampling a minibatch for each update

Require: Learning rate ϵ
Require: Initial parameter $\boldsymbol{\theta}$
  while stopping criterion not met **do**
    Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
    Compute gradient estimate: $\hat{\boldsymbol{g}} \leftarrow +\frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$.
    Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \hat{\boldsymbol{g}}$.
  end while

Batch Size

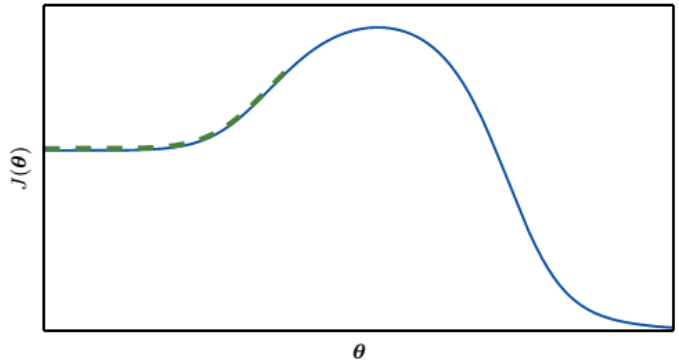
- Larger size: more accurate, less than linear return, more memory
- Smaller size: has regularizing effect, increasing processing overhead due to small learning rate for stability

Challenges for Deep Learning Optimization

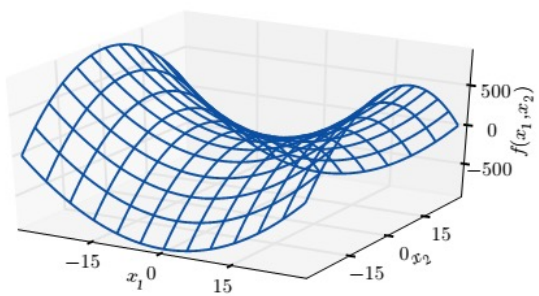


Challenges for Deep Learning Optimization

- Convex optimization
  - Local minimum is global minimum
- Nonconvex optimization
  - Many local minima



- Deep models are nonconvex functions
  - Weight space symmetry
  - Multiple equivalently parametrized latent variables
- Saddle point
  - First-order method
    - Gradient becomes small
    - Can escape in many cases
  - Newton's method
    - Trapped into saddle point

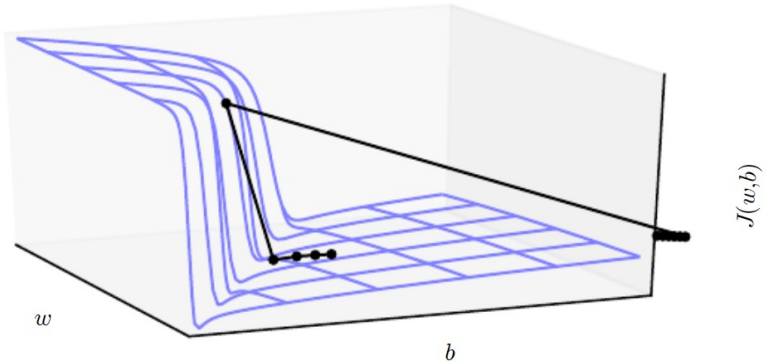


More on Challenges

- Deep learning often involves multiplication of weights

$$W = V\mathrm{diag}(\lambda)V^{-1}$$
$$W^t = V diag(\lambda)^t V^{-1}$$

- Cliff and exploding gradients
  - Multiplication of several large weights
  - Move step too fast
  - Can be avoided using gradient clipping
- Vanishing gradients
  - Small weights



Stochastic Gradient Descent

- SGD with decreasing learning rate

Require: Learning rate ϵ_k
Require: Initial parameter $\boldsymbol{\theta}$
while stopping criterion not met **do**
Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
Compute gradient estimate: $\hat{\boldsymbol{g}} \leftarrow +\frac{1}{m}\nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$.
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon_k \hat{\boldsymbol{g}}$.
end while

  - Condition to guarantee convergence

$$\sum_{k=1}^{\infty} \epsilon_k = \infty, \sum_{k=1}^{\infty} \epsilon_k^2 < \infty,$$

  - Practical choice: decay linearly until τ and then constant

$$\epsilon_k = (1-\alpha)\epsilon_0 + \alpha\epsilon_\tau, \alpha = \frac{k}{\tau}$$

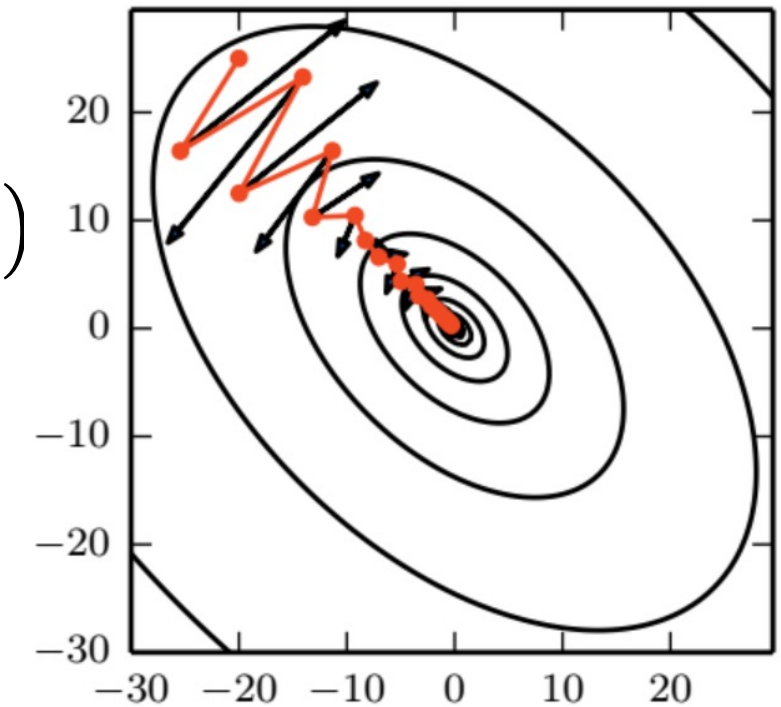
Momentum

- SGD with Momentum
  - High curvature or noisy gradients
  - Smoothing gradients
- Introduce a velocity v

$$v \leftarrow \alpha v - \epsilon \nabla_\theta \left(\frac{1}{M} \sum_{i=1}^{M} l(f(x_i; \theta), y_i) \right)$$

$$\theta \leftarrow \theta + v$$

- Adaptive learning rates
  - AdaGrad
  - RMSProp
  - Adam



Initialization

- Simple and heuristic
- Break symmetry between different units
- Set weights to small and random values
- Set biases to constants

Summary

- Key concepts

Deep learning, Feedforward Networks, Depth, Width, Hidden Variables, Universal Approximation Theorem

- Models

GNN (MLP), Bayesian Estimator

- Algorithms

Forward Propagation, Backward Propagation, Adaptive Learning Rates

References

- *Deep Learning*, I. Goodfellow, Y. Bengio, and A. Courville, MIT Press, 2016
- *Python Machine Learning,* S. Raschka and V. Mirjalili, Packt Publishing, 3rd Edition, 2019